Rapport DataOps d'Ingénierie des Données

Du Script Fragile à l'ELT Moderne avec DuckDB, dbt, Git et DVC — TP 1, TP 2 & TP 3

Auteurs : Équipe DataOps	**Environnement :** Python 3.13.17 | DuckDB 2.0 | dbt 1.12	**Statut global :** 100% VALIDÉ
Date : Novembre 2026	**Dépôt :** dataops-lab1 (Git + DVC)	**Certification :** 16/16 tests dbt | 6/6 tests pytest

1. Synthèse Exécutive et Tableau Récapitulatif

Ce projet documente la transformation progressive et industrielle d'un processus de traitement de données pour une boutique en ligne. Parti d'un script Python naïf présentant des failles critiques (erreurs silencieuses, surestimation de +66.8% du CA), le projet a évolué vers un pipeline modulaire testé (TP1), versionné au niveau code et données avec Git et DVC (TP2), puis intégralement réarchitecturé selon le paradigme ELT moderne avec un entrepôt analytique DuckDB et dbt (TP3).

Travail Pratique	Objectif DataOps	Outils Clés	Indicateur / Résultat
TP 1 — Script fragile vers pipeline testé	Structuration en fonctions, contrôles de schéma, arrêt de chaîne, quarantaine.	Python, Pandas, Pytest	**TOTAL: 7 655,90 €** 5 tests unitaires passés J2/J3 refusés (arrêt net)
TP 2 — Versionner code (Git) & données (DVC)	Branches de fonctionnalités, Content-Addressable Storage, tags de données, reproductibilité.	Git, DVC, Remote Storage	**TOTAL: 8 835,40 €** Tags data-v1 & data-v2 Clone collègue validé
TP 3 — Pipeline ELT moderne (DuckDB + dbt)	Chargement brut tel quel, transformation SQL en couches, tests singuliers, tests unitaires dbt.	DuckDB, dbt Core, Jinja, SQL	**PASS=14 (PASS=16)** Total exact: **8 835,40 €** Lignage automatique généré

2. TP 1 — D'un Script Fragile à un Pipeline Testé

Problématique initiale : Le script naïf (naive_report.py) calcule quantity * unit_price sans aucun contrôle. Il affichait un chiffre d'affaires erroné de **12 771,10 €** au lieu des **7 655,90 €** réels (surestimation de +66.8%) en raison d'anomalies non détectées.

Tableau des anomalies interceptées dans orders_day1.csv :

Dimension de Qualité	Order ID	Description du Problème	Action / Règle appliquée
Complétude	1008	customer_id absent (valeur vide)	Rejeté : missing_customer_id
Unicité	1010	Commande dupliquée à l'identique	Rejeté : duplicate_order_id
Validité	1004	Quantité négative (quantity = -2)	Rejeté : invalid_quantity
Validité	1013	Date impossible (2026-13-45)	Rejeté : invalid_date
Exactitude / Plausibilité	1019	Clavier à 4 500 € au lieu de 45 €	Rejeté : invalid_unit_price
Cohérence	1006, 1016, 1023	Casse et espaces (' tunisia', 'MOROCCO')	Nettoyé : .str.strip().str.title()

Réponses aux Questions Clés du TP1 :

Q1 (requirements.txt) : Figer les dépendances et versions minimales assure la reproductibilité stricte sur toute machine et dans la CI via pip install -r requirements.txt.

Q4 (Évaluation du total sans contrôle) : Impossible de savoir si le chiffre naïf est trop haut ou trop bas car les erreurs se compensent partiellement (les quantités négatives tirent vers le bas, les doublons et prix aberrants tirent vers le haut).

Q8 (Fichier de quarantaine) : Enregistrer les lignes rejetées dans rejected_orders.csv garantit l'observabilité et l'auditabilité : les équipes métier peuvent investiguer et retraiter les commandes légitimes sans perte d'information.

Q13 (Danger J2 vs J3) : Le Jour 2 est infiniment plus dangereux car l'ancien script tourne sans erreur et publie silencieusement un total colossalement faux (270 200 € au lieu de 2 702 €). Le Jour 3 plante bruyamment (KeyError), ce qui alerte immédiatement.

Q14 (Code de sortie 1) : Utilisé par les orchestrateurs (Airflow, Dagster) et la CI (GitHub Actions) pour bloquer l'exécution en aval et alerter l'équipe en cas d'anomalie.

3. TP 2 — Versionner le Code avec Git et les Données avec DVC

Objectif : Les pipelines de données dépendent de deux composants qui évoluent indépendamment : le code et les données. Les données volumineuses ne doivent pas être commitées dans Git (gonflement du dépôt). DVC permet de versionner les données dans un stockage externe tout en conservant un pointeur léger (.dvc) tracé par Git.

Architecture et Cycle de Vie Mis en Œuvre :

1. **Branche de fonctionnalité :** Création de feature/country-rule pour la Règle 6 (rejet des pays hors périmètre avec unknown_country), couplée à un test pytest (6/6 passés), puis fusion explicite git merge --no-ff sur main.
2. **Initialisation DVC :** Suppression des CSV de l'index Git (git rm --cached) et suivi par DVC (dvc add data/raw). Génération de data/raw.dvc (empreinte MD5) et tag data-v1.
3. **Remote de stockage :** Configuration de ../dvc-storage et synchronisation via dvc push (Content-Addressable Storage).
4. **Nouvelle version de données :** Intégration de 5 commandes tardives (35 commandes au total). Mise à jour de l'empreinte MD5, création du tag data-v2. Le pipeline produit le nouveau chiffre d'affaires exact : **8 835,40 €**.
5. **Time-Travel vérifié :** Restauration de l'état passé via git checkout data-v1 -- data/raw.dvc et dvc checkout (retour à 31 lignes et 7 655,90 €), puis retour immédiat à HEAD.
6. **Reproduction par un tiers :** Clone simulé dans ../teammate : git clone + dvc pull reproduit exactement les **8 835,40 €**.

Réponses aux Questions Clés du TP2 :

Q1 (Isolation des branches) : Pendant le travail sur la branche, main reste totalement stable. Si un batch tourne la nuit depuis main, il ne risque jamais d'exécuter un code en cours de test.

Q3 & Q5 (Empreinte MD5 et nommage CAS) : Les fichiers dans dvc-storage sont nommés par leur hash MD5. Ce nommage par contenu évite les collisions, déduplique automatiquement les fichiers identiques et permet à plusieurs versions de coexister.

Q9 (Oubli de dvc checkout) : Si l'on change de version dans Git sans exécuter dvc checkout, les fichiers physiques sur disque ne correspondent plus au pointeur. Le pipeline s'exécute sur des données désynchronisées, produisant un rapport faux tout en prétendant correspondre à l'ancien commit.

Q10 (Rôles respectifs de Git et DVC) : git clone transfère le code, l'historique et les pointeurs légers ; dvc pull télécharge les volumineux fichiers de données depuis le stockage distribué.

Graphe Git des branches, fusions et tags DVC :

```
* 7e3671c Describe the report as a DVC stage
* 95a8f79 (tag: data-v2) Data v2: late orders for 2026-10-05
* 454c740 Add the DVC remote
* 840c0e0 (tag: data-v1) Track raw data with DVC
* af9ef9a Merge the country rule
|\
| * 010fdc6 (feature/country-rule) Reject orders from countries the shop does not sell to
|/
* caa0628 Finish TP1, add DVC to requirements
* 4cd79aa Add pipeline, data checks and unit tests
* 3c8775a Empty project with requirements
```

4. TP 3 — Du Pipeline Python à l'ELT Moderne avec DuckDB et dbt

Transition ETL vers ELT : Dans l'ETL (TP1), la transformation s'exécutait en mémoire Python avant écriture. Dans l'ELT moderne (TP3), le fichier brut est chargé tel quel dans l'entrepôt analytique (DuckDB), et l'ensemble des transformations est orchestré en SQL en couches modulaires via dbt (Data Build Tool).

Architecture en Couches dbt Implémentée :

Couche	Modèle SQL	Matérialisation	Rôle & Spécificités DataOps
Source	raw.orders	Table brute	Chargé par load_raw.py avec all_varchar=true. Colonnes techniques _row_number, _source_file, _loaded_at.
Staging	stg_orders	View	Typage strict (try_cast, try_strptime, decimal(10,2)). Nettoyage de casse/espaces des pays. **Aucune ligne supprimée**.
Intermediate	int_orders_checked	View	Application séquentielle des **6 règles de qualité** via CASE WHEN ... END AS reject_reason.
Marts	orders_clean orders_rejected revenue_by_country	Tables	• orders_clean : commandes saines (reject_reason IS NULL). • orders_rejected : table de quarantaine. • revenue_by_country : rapport direction (**8 835,40 €**).
Bonus Marts	revenue_by_product	Table	Nouvel indicateur analyste ajouté sans écrire une seule ligne de Python.

Batterie de Tests dbt (Tests de Données & Tests Unitaires) :

• **Tests génériques (marts.yml) :** unique et not_null sur order_id, not_null sur customer_id, accepted_values sur country.
• **Tests singuliers (SQL métier) :** assert_quantity_positive.sql (quantités > 0) et assert_reject_rate_below_20_percent.sql (arrêt si taux rejet > 20%).
• **Tests unitaires dbt (int_orders_checked.yml) :** Vérification de la logique SQL sur des données synthétiques injectées en YAML (rejet de pays non autorisés et déduplication).
• **Résultat d'exécution :** Done. PASS=14 WARN=0 ERROR=0 SKIP=0 (et PASS=16 avec le modèle analyste).

Validation de la Résistance aux Mauvaises Données (Jours 2 et 3) :

Scénario	Comportement Pipeline Python (TP1)	Comportement Pipeline dbt (TP3)
Jour 2 : Prix aberrants en centimes (multipliés par 100)	check_reject_rate lève DataQualityError (100% rejet). Arrêt net (exit code 1).	Test singulier assert_reject_rate_below_20_percent en **FAIL**. Tous les marts passent en **SKIP** (exit code 1). Rapport J1 préservé.
Jour 3 : Renommage de colonne source ('price')	check_schema lève DataQualityError: Missing columns: ['unit_price']. Arrêt net.	Échec immédiat de compilation/liaison DuckDB dans stg_orders (Binder Error). 13 étapes suivantes en **SKIP** (exit code 1).
Retour Jour 1 : Données saines	Succès complet, total = 8 835,40 €.	**PASS=14** (100% vert), total exact = **8 835,40 €**.

5. Synthèse des Bonnes Pratiques DataOps Validées

Pilier DataOps	Principe Théorique	Mise en Pratique dans le Projet
Analytics is Code	Toute logique de données doit être versionnée, revue et testable comme du logiciel.	Transformations en fichiers SQL versionnés dans Git, dépendances déclarées dans dbt_project.yml.
Data as Code	Séparer le stockage des données volumineuses du code source tout en garantissant leur traçabilité.	Données gérées par DVC (Content-Addressable Storage), pointeurs légers .dvc suivis dans Git.
Tests de Code vs Tests de Données	Distinguer la validité de la logique algorithmique de la conformité des données entrantes.	• Tests de code : Pytest & tests unitaires dbt (données mockées statiques). • Tests de données : contrôles de schéma, tests singuliers dbt (données réelles de production).
Arrêter la Chaîne (Andon Cord)	Ne jamais publier de rapport sur des données corrompues ou incomplètes.	Échec bloquant de dbt build mettant les tables aval en SKIP et préservant le rapport de la veille.
Observabilité & Lignage	Comprendre immédiatement l'impact d'une modification et tracer les rejets.	Graphe de dépendance DAG généré avec dbt docs et table de quarantaine matérialisée orders_rejected.
Intégration Continue (CI)	Exécution automatique des tests à chaque push ou Pull Request.	Workflow GitHub Actions configuré dans .github/workflows/ci.yml.

Conclusion :

Ce cycle de trois TP démontre avec rigueur comment transformer un script artisanal en une plateforme d'ingénierie de données robuste, collaborative et pérenne. L'équipe dispose désormais d'un pipeline ELT moderne, d'un entrepôt analytique autonome, d'une couverture complète de tests de qualité, et d'un couplage parfait entre l'historique du code et celui des données.

Document généré automatiquement dans le cadre du projet dataops-lab1 — Tous droits réservés.